BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS
ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

GCC PLUGIN FOR STATIC ANALYZER SUPPORT

GCC PLUGIN PRO STATICKÉ ANALYZÁTORY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **LUKÁŠ PŠEJA**
AUTOR PRÁCE

SUPERVISOR **Dr. Ing. PETR PERINGER,**
VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Bachelor's Thesis Assignment

172017

Institut:	Department of Intelligent Systems (DITS)
Student:	**Pšeja Lukáš**
Programme:	Information Technology
Title:	**GCC plugin for static analyzer support**
Category:	Software analysis and testing
Academic year:	2025/26

Assignment:

1. Get acquainted with *CodeListener* component from the *Predator* project. Analyze the implementation of *CodeListener* and similar projects. Additionally, study the relevant data structure visualization concepts.
2. Design a new GCC plugin as a modern alternative to *CodeListener* and add new functionality (built-in visualization support and export in JSON format). Focus on improvement of the plugin interface.
3. Implement the plugin in C++ along with appropriate examples for demonstration purposes. Verify the plugin's compatibility by using it with *Predator*.
4. Test the plugin using the created examples with various GCC versions. Evaluate obtained results.

Literature:

- GitHub repository of Predator project. https://github.com/kdudka/predator

Requirements for the semestral defence:

- First two points of assignment.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor:	**Peringer Petr, Dr. Ing.**
Head of Department:	Kočí Radek, Ing., Ph.D.
Beginning of work:	1.11.2025
Submission deadline:	13.5.2026
Approval date:	3.11.2025

Abstract

This bachelor's thesis deals with the analysis of the Code Listener infrastructure, which serves as the foundation for creating static analyzers, and its subsequent refactoring into a new plugin.

Abstrakt

Tato bakalářská práce se zabývá analýzou infrastruktury Code Listener, která poskytuje základ pro tvorbu statických analyzátorů zdrojového kódu, a její následnou refaktorizací do nového pluginu.

Keywords

Code Listener infrastructure, GCC, LLVM, plugin, refactoring, static analysis, C, C++

Klíčová slova

infrastruktura Code Listener, GCC, LLVM, plugin, refaktorování, statická analýza, C, C++

Reference

PŠEJA, Lukáš. *GCC plugin for static analyzer support.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Dr. Ing. Petr Peringer,

GCC plugin for static analyzer support

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Dr. Ing. Petr Peringer. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used Grammarly to correct spelling and other language mistakes.

.....
Lukáš Pšeja
November 30, 2025

Contents

List of Figures

Chapter 1

Introduction

With the ever-increasing complexity of software systems and the growing demand for high-quality and secure code, formal verification methods have become essential in some parts of the software development industry [**?**]. Formal verification methods are a set of mathematical techniques used to prove the correctness of software systems, going beyond traditional testing and debugging methods, which only test a finite number of inputs, allowing potentially critical errors to go undetected [**?**]. The implementation of such formal methods is not straightforward, as it requires a deep understanding of both the verified software and the application of formal methods, typically necessitating specialized domain knowledge. Formal verification tools, such as Predator or Forester [**?**, **?**], utilize static analysis to verify code written by developers. In Predator's case, this involves verifying sequential C programs for memory safety while operating with pointers and linked lists [**?**]. Predator is built on top of the Code Listener infrastructure [**?**], which provides an application programming interface (API) foundation for creating such static analyzers.

However, the core of the Code Listener infrastructure was implemented nearly fifteen years ago as a part of Predator, which has led to certain challenges in continuing its development and maintenance in its current form. Consequently, the codebase relies on outdated programming practices and lacks the expressiveness, safety, and efficiency offered by modern C++ standards. Moreover, this legacy technical debt, combined with the complexity of the existing codebase, hinders further development and makes maintenance and adaptation to newer GCC versions more difficult, as well as limits the potential for reusability in other static analysis tools.

To address these challenges, this bachelor's thesis focuses on refactoring the Code Listener infrastructure into a standalone, modern GCC plugin [**?**]. The primary objective is to design and implement a new architecture that serves as a robust alternative to the existing solution. By leveraging the capabilities of modern C++ standards, targeting C++23, the new implementation aims to simplify the internal structure, unify the API, and ensure overall long-term maintainability. Furthermore, the modernized structure will allow for easier integration of new functionality, specifically the support for data structure visualization and export to JSON format.

The thesis is structured as follows. Chapter 2 provides an overview of the necessary background information, including TODO: names of the chapters

Chapter 2

Preliminaries

To understand the work presented in this thesis, it is essential to have a foundational understanding of several key concepts described in this chapter.

2.1 Static program analysis

TODO: Static program analysis

2.2 Intermediate Representation

TODO: Intermediate Representation

2.2.1 Three-Address Code

TODO: Three-Address Codend later

2.2.2 Static Single Assignment (SSA)

TODO: Static Single Assignment (SSA)

2.3 GNU Compiler Collection

The GNU Compiler Collection (GCC) is a compiler developed by the Free Software Foundation, Inc. (FSF) under the GNU General Public License (GPL), initially written for the GNU operating system, also developed by the FSF. It has since become a widely used compiler for various operating systems, platforms [1] and programming languages, such as C, C++, Objective C, Objective C++, Fortran, Ada, Go, D, Modula-2, Rust, and Cobol. GCC is developed and maintained by a large community of contributors worldwide, with major decisions about GCC being made by the steering committee [**?**].

2.3.1 Internal Structure

The internal structure of GCC is designed with a strong emphasis on retargetability, allowing the compiler to support multiple input languages and target diverse hardware architec-

[1] The list of supported platforms is available here: https://gcc.gnu.org/install/specific.html and should be checked before installing on your system.

tures without requiring a complete rewrite of the entire codebase [**?**]. To achieve this, GCC is divided into three main components that communicate through intermediate code.

TODO: draw a graph here?

Front-End

The Front-End is responsible for processing the source code written in a specific high-level language. What a high-level language is is subjective, but in this thesis, we consider languages that abstract away low-level hardware details and provide constructs that are more human-readable, such as C or C++.

Its primary tasks include lexical analysis, syntax analysis, semantic analysis, and the generation of an intermediate code. The output of the Front-End is a language-independent representation called *GENERIC*. GENERIC is a tree-based structure that preserves the high-level constructs of the original code but abstracts away the syntactic specifics of the source language [**?**].

This abstraction allows the subsequent phases of the compiler to be language-agnostic.

Middle-End

The Middle-End serves as the core machine-independent optimization engine of GCC. It works on an intermediate representation called *GIMPLE* derived from GENERIC. GIMPLE has two kinds: *High-level GIMPLE* and *Low-level GIMPLE*. The derivation process involves lowering the GENERIC representation into High-level GIMPLE. High-level GIMPLE is converted into low-level GIMPLE by linearizing complex control flow constructs into simpler forms, such as nested functions, exception handling, and loops. Lastly, the Low-level GIMPLE is rewritten into SSA form for further optimizations [**?**].

The plugin developed in this thesis operates at this level.

Back-End

The Back-End translates the optimized SSA GIMPLE representation into a Lisp-like language called the *Register Transfer Language (RTL)*. RTL is a low-level intermediate representation that is closely aligned with the assembly language of the target architecture. The Back-End performs target-specific optimizations, including but not limited to instruction scheduling and register allocation, before finally emitting the assembly code for the specific hardware [**?**].

This modular structure ensures that adding support for a new programming language requires implementing only a new Front-End, while supporting a new processor architecture requires implementing only a new Back-End, leaving the Middle-End unchanged.

2.3.2 Plugins

TODO: GCC plugins

2.4 Predator

TODO: Predator

2.5 Control Flow Graph

TODO: Control Flow Graph

2.6 The Code Listener Infrastructure

The Code Listener infrastructure was designed to simplify the implementation of static analyzers for C programs. Its primary purpose is to abstract the communication with the compiler (specifically GCC) during the compilation process. This allows the infrastructure to process any source code accepted by the compiler [**?**]. The infrastructure translates the compiler's internal representation into a normalized format and provides it to the static analyzer via an object-oriented API called Code Storage or through various optional output modules. The high-level context is visualized in figure 2.1 on page 6.

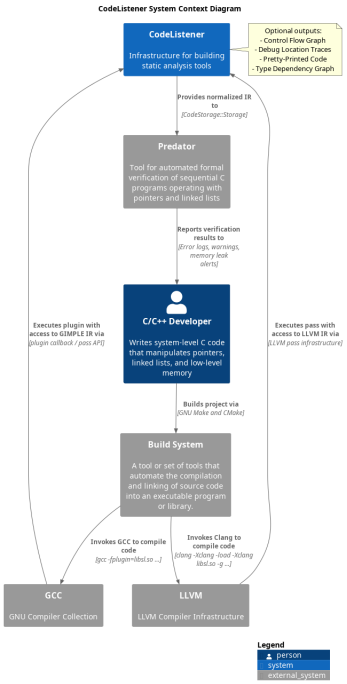

Figure 2.1: Code Listener system context diagram

The Code Listener infrastructure consists of several components visualized in figure 2.3 on page 8, that work together to achieve the aforementioned goals.

Adapters are the components that are embedded into the code parsers (e.g., GCC or Sparse [2]) and are responsible for translating the internal representation (IR) of the specific code parser into a normalized format, which provides a consistent way to represent the compiled program regardless of the code parser used.

The *code parser interface* represents the API used for communication between GCC or other code parsers and Code Listener during the compilation process. This API relies on a callback mechanism, where the adapter emits events for constructs of the intermediate code while traversing the parser's internal data structures. These callbacks are then processed by *filters* and *listeners*.

[2] https://sparse.docs.kernel.org/

The data flowing through this interface represents the program as a *Control Flow Graph (CFG)*. The CFG consists of basic blocks containing a sequence of instructions. The infrastructure simplifies the complex GIMPLE representation into a concise instruction set, distinguishing between *terminal instructions* (e.g., jumps, returns) and *non-terminal instructions* (e.g., assignments, calls) [**?**].

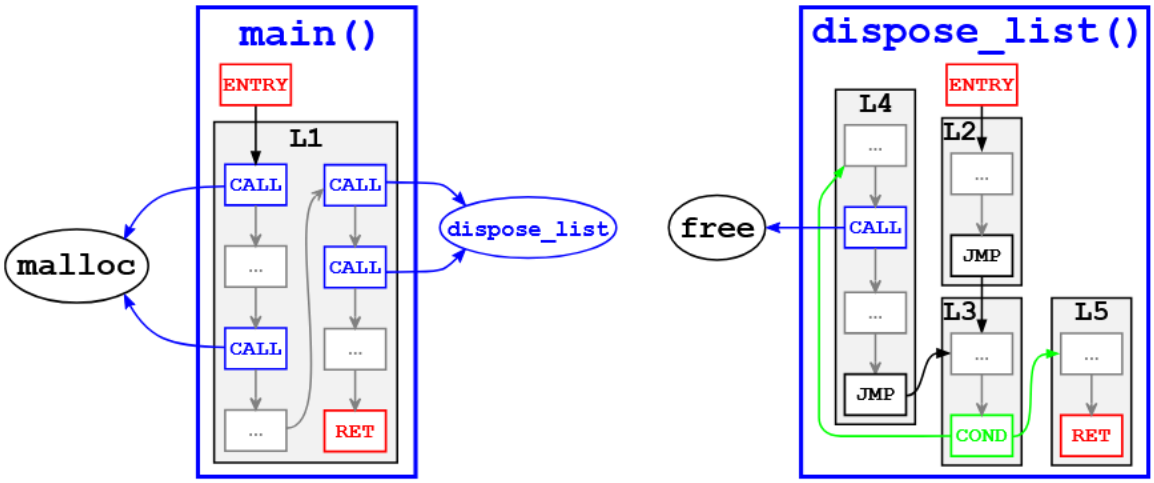

Figure 2.2: Control Flow Graph basic blocks[3]

Filters enable the transformation of the intermediate code emitted by the adapters. They operate as a middleware layer that can normalize or optimize the instruction stream before it reaches the storage. A notable example is the *switch to if* filter, which transforms `SWITCH` instructions into a sequence of conditional `COND` jumps, simplifying the analysis for downstream tools [**?**].

In contrast to filters, *listeners* are consumers of the API. They register handlers for specific callbacks to process the intermediate code. Standard listeners included in the infrastructure serve primarily as diagnostic tools, such as the *CFG plotter*, which generates graph visualizations, or the code dumper for the three-address code representation [**?**].

Finally, the *Code Storage* component builds a persistent, in-memory object model of the program from the stream of callbacks. Unlike the transient nature of callbacks, Code Storage allows the static analyzer to perform random-access queries on the program structure, types, variables, and functions once the object model is fully constructed [**?**].

2.6.1 Legacy Implementation Analysis

The existing infrastructure is implemented using the C++03 standard. Consequently, it lacks many features and safety improvements introduced in modern C++ revisions. The object model relies heavily on raw pointers and C-style structures, which are exposed directly to the C interface, thereby ignoring modern encapsulation principles. Memory management is largely manual or implicit, lacking the guarantees provided by modern smart pointers (RAII idiom). This technical debt makes the codebase prone to memory leaks and difficult to extend or maintain in the context of rapidly evolving GCC versions.

[3]Taken from [**?**].
[4]Taken from [**?**].

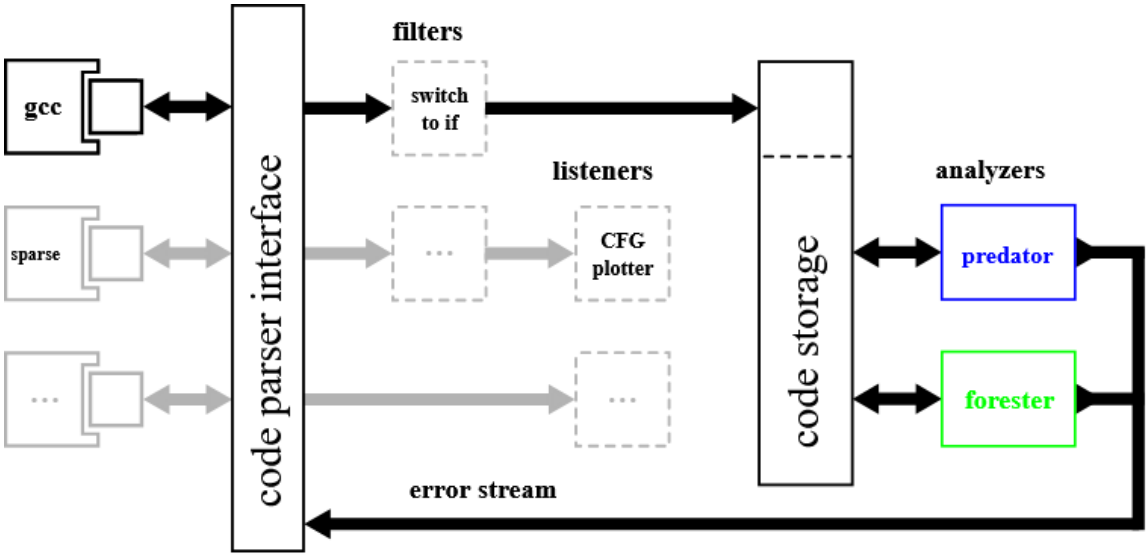

Figure 2.3: Code Listener infrastructure[4]

This thesis aims to address these challenges by refactoring the infrastructure into a standalone, modern GCC plugin. The new implementation utilizes C++23 features to ensure type safety, robust resource management, and long-term maintainability.

2.7 Modern C++ Practices

TODO: Modern C++ Practices

Chapter 3

Design

TODO: design

Chapter 4

Implementation

TODO: implementation

Chapter 5

Testing

TODO: testing

Chapter 6

Conclusion

TODO: conclusion

Bibliography